

LABORATORY OBSERVATION

Course	Full Stack Web Development
Course Code	23CM4121
Experiment No.	6
Student Name	Kavya G.
Roll No.	A24126552082
Date	23-09-2026

1. TITLE

Node.js Modules – os, path and fs Modules

2. AIM

To understand Node.js modules and demonstrate the purpose of the built-in os, path, and fs modules with suitable examples.

3. OBJECTIVE

- To understand the concept of modularity in Node.js.
 - To learn how built-in Node.js modules are imported and used.
 - To obtain operating-system information using the os module.
 - To work with file and directory paths using the path module.
 - To perform file handling operations using the fs module.
-

4. THEORY

A Node.js module is a separate piece of JavaScript code that provides specific functionality. Node.js provides several built-in modules that can be used directly without installing additional packages.

The os module is used to obtain information about the operating system. The path module provides functions for working with file and directory paths. The fs (File System) module is used to create, read, write, append, and manage files and directories.

5. MODULES USED

Module	Purpose	Example Function
os	Provides operating-system information.	os.platform()
path	Works with file and directory paths.	path.join()

fs	Performs file-system operations.	fs.readFileSync()
----	----------------------------------	-------------------

6. ALGORITHM / PROCEDURE

1. Import the os, path, and fs modules using require().
2. Use the os module to display the operating system, CPU architecture, home directory, and number of CPUs.
3. Use the path module to create a file path.
4. Use path.basename() to obtain the file name.
5. Use path.extname() to obtain the file extension.
6. Use the fs module to read the contents of the file.
7. Display the file content in the terminal.

7. PROGRAM

```
// os module
const os = require('os');

console.log("Operating System: " + os.platform());
console.log("CPU Architecture: " + os.arch());
console.log("Home Directory: " + os.homedir());
console.log("Number of CPUs: " + os.cpus().length);

// path module
const path = require("path");

const filePath = path.join("students", "data.txt");

console.log("File Path: " + filePath);
console.log("File Name: " + path.basename(filePath));
console.log("Extension: " + path.extname(filePath));

// fs module
const fs = require("fs");
fs.writeFileSync("message.txt", "Hello from Node.js!");
const data = fs.readFileSync(message.txt, "utf8");
console.log("File Content: " + data);
```

8. CODE EXPLANATION

Code / Function	Explanation
require('os')	Imports the built-in os module.
os.platform()	Returns the operating-system platform.
os.arch()	Returns the CPU architecture.

os.homedir()	Returns the user's home directory.
os.cpus().length	Returns the number of CPU cores available.
require('path')	Imports the path module.
path.join()	Combines path parts into a file path.
path.basename()	Returns the file name from a path.
path.extname()	Returns the extension of a file.
require('fs')	Imports the File System module.
fs.readFileSync()	Reads the contents of a file synchronously.

9. OUTPUT

```

PS D:\kavya\Tutorials\FS lab\Observation_Tasks\task 6\nodejs_modules> node code.js
Operating System: win32
CPU Architecture: x64
Home Directory: C:\Users\HP15s-fr2005tu
Number of CPUs: 8
File Path: students\data.txt
File Name: data.txt
Extension: .txt
File Content: Hello from Node.js!
PS D:\kavya\Tutorials\FS lab\Observation_Tasks\task 6\nodejs_modules>

```

10. RESULT

Thus, the concept of Node.js modules was studied successfully. The os module was used to obtain operating-system information, the path module was used to handle file paths, and the fs module was used to read file contents successfully.